

Лабораторная №10. Основы программирования микроконтроллеров Arduino

Цель работы

Освоить базовые принципы программирования микроконтроллеров на примере платформы Arduino, научиться управлять светодиодами и обрабатывать внешние прерывания, а также организовать совместную работу Arduino и одноплатного компьютера.

Оборудование и исходные данные

- Arduino Uno и USB-кабель;
- светодиодная мезонинная плата;
- Lichee RV Dock с программой генерации меандра на GPIO 144 из лабораторной №9;
- преобразователь логических уровней $5 \leftrightarrow 3,3$ В на макетной плате;
- соединительные провода;
- компьютер с Arduino IDE и PulseView;
- логический анализатор.

Ключевые понятия

Платформа Arduino

Определение
Arduino — открытая аппаратно-программная платформа для прототипирования электронных устройств на базе микроконтроллерных плат и среды разработки.

- **микроконтроллер** — центральный вычислительный компонент платы, для Arduino Uno это ATmega328P;
- **программная среда** — Arduino IDE с библиотеками для работы с периферией;
- **стандартизированные разъёмы** — контакты для подключения периферии;
- **открытая архитектура** — возможность изучения, модификации и создания совместимых плат.

Программирование микроконтроллера и разработка под ОС

Программирование микроконтроллеров, таких как Arduino, принципиально отличается от разработки под операционные системы. В Arduino отсутствует операционная система — программа работает напрямую с аппаратным обеспечением, что обеспечивает прямой доступ к регистрам

периферийных устройств. Это позволяет управлять оборудованием на самом низком уровне, но требует от разработчика глубокого понимания архитектуры микроконтроллера.

Одним из важных преимуществ программирования микроконтроллеров является детерминированное выполнение операций — время выполнения каждой инструкции известно точно, что критически важно для систем реального времени. Однако это достигается за счёт ограниченных ресурсов: микроконтроллеры имеют мало памяти (обычно килобайты) и работают на сравнительно низких частотах процессора (мегагерцы вместо гигагерц).

Определение

Прошивка (firmware) — программа, записанная в память микроконтроллера и запускаемая при включении или сбросе платы. Приложение под Linux хранится как исполняемый файл и запускается или останавливается средствами операционной системы.

В отличие от этого, разработка под операционную систему, как на Lichee RV Dock, происходит на другом уровне абстракции. Здесь программа работает поверх ядра Linux, которое предоставляет абстракцию оборудования через драйверы и системные вызовы. Это упрощает разработку, но добавляет накладные расходы. Операционная система обеспечивает многозадачность — несколько программ могут работать одновременно, разделяя ресурсы процессора. Системы с ОС обычно имеют значительно больше ресурсов: больше памяти, более высокую производительность процессора и развитую файловую систему. Программа в таком контексте представляет собой исполняемый файл, который можно запускать и останавливать по мере необходимости.

Архитектура микроконтроллера ATmega328P

Arduino Uno основана на микроконтроллере ATmega328P: - **Тактовая частота:** 16 МГц - **Флэш-память:** 32 КБ (для программы) - **ОЗУ:** 2 КБ (для данных) - **Энергонезависимая память:** 1 КБ EEPROM - **Периферия:** таймеры, АЦП, UART, SPI, I2C, GPIO

Загрузка прошивки в Arduino

Процесс программирования Arduino состоит из нескольких этапов. Сначала разработчик пишет код в среде Arduino IDE, используя упрощённый диалект C/C++ с готовыми библиотеками для работы с периферией. Затем среда разработки компилирует код в машинные инструкции для архитектуры AVR, оптимизируя его под ограниченные ресурсы микроконтроллера.

После компиляции происходит загрузка прошивки через USB-порт с использованием встроенного программатора. Как только прошивка успешно загружена, программа начинает выполняться сразу же — микроконтроллер переходит к выполнению кода без необходимости перезагрузки или дополнительных действий.

Определение

Загрузчик (bootloader) — служебная программа в памяти микроконтроллера, которая принимает новую прошивку и передаёт управление основной программе.

При включении Arduino загрузчик первым делом проверяет, не поступила ли команда на загрузку новой прошивки. Если такой команды нет, он просто передаёт управление основной программе. Если же пользователь пытается загрузить новый скетч, загрузчик активирует режим программирования: он принимает данные через последовательный порт (UART), проверяет их целостность, и записывает во флэш-память микроконтроллера. После успешной записи загрузчик выполняет верификацию — сравнивает записанные данные с полученными, и только затем запускает новую программу. Эта система позволяет программировать Arduino без использования внешних программаторов, что значительно упрощает процесс разработки и отладки.

Основные концепции программирования Arduino

Определение

Скетч (sketch) — исходная программа для Arduino. Её структура включает функции `setup()` и `loop()`.

```
void setup() {  
    // Выполняется один раз при старте  
    // Инициализация пинов, настройка периферии  
}  
  
void loop() {  
    // Выполняется бесконечно в цикле  
    // Основная логика программы  
}
```

Определение

Цифровые входы и выходы (GPIO) — контакты общего назначения, режим которых задаётся функцией `pinMode()`.

Базовые режимы:

- **OUTPUT** — пин работает как выход: микроконтроллер управляет напряжением на пине (HIGH — 5 В или LOW — 0 В) для управления светодиодами, реле и другими устройствами
- **INPUT** — пин работает как вход: микроконтроллер считывает внешний сигнал (HIGH или LOW) от кнопок, датчиков и других схем

Помимо базовых, существуют дополнительные режимы, которые настраивают внутренние резисторы микроконтроллера:

- **INPUT_PULLUP** — вход с подтяжкой к питанию. Внутренний резистор (около 20–50 кОм) подтягивает пин к высокому уровню (HIGH). Когда внешнее устройство (например, кнопка) соединяет пин с землёй, читается LOW. Этот режим удобен для кнопок и контактов — не нужен внешний резистор, а по умолчанию пин всегда находится в известном (высоком) состоянии

Определение

INPUT_PULLDOWN — режим входа с внутренней подтяжкой к земле, доступный не на всех микроконтроллерах. ATmega328P на Arduino Uno его не поддерживает: используйте внешний резистор или INPUT_PULLUP с инвертированной логикой.

Определение

Прерывание (interrupt) — механизм, при котором процессор временно приостанавливает основной код и выполняет обработчик в ответ на аппаратное или программное событие.

После завершения обработчика процессор продолжает выполнение прерванного кода.

Различают два основных типа прерываний. **Аппаратные прерывания** возникают от внешних устройств и периферии микроконтроллера — сигналов на цифровых пинах, таймеров, UART, SPI, I2C и других коммуникационных интерфейсов. **Программные прерывания** инициируются самим кодом через специальные инструкции процессора — они используются для вызова функций операционной системы или приоритетного переключения задач.

Режимы срабатывания аппаратных прерываний: При настройке внешнего прерывания на пине можно выбрать один из четырёх режимов: - **LOW** — прерывание срабатывает постоянно, пока на пине низкий уровень - **CHANGE** — прерывание срабатывает при любом изменении сигнала (с HIGH на LOW и наоборот) - **RISING** — прерывание срабатывает только при переходе с LOW на HIGH (восходящий фронт) - **FALLING** — прерывание срабатывает только при переходе с HIGH на LOW (нисходящий фронт)

На Arduino Uno внешние прерывания доступны на пинах 2 и 3.

Внимание

Обработчик прерывания (ISR) должен завершаться как можно быстрее. Не вызывайте в нём `delay()`, `Serial.print()` и другие блокирующие или зависящие от прерываний функции. Переменные, совместно используемые ISR и основным кодом, объявляйте как `volatile`; для многобайтовых данных дополнительно обеспечивайте атомарный доступ.

Порядок выполнения

Подготовка оборудования

1. **Arduino Uno** — подключите к компьютеру через USB
2. **Светодиодная мезонинная плата** — вставляется в соответствии с распиновкой Arduino
3. **Соединительные перемычки** — для соединения оборудования
4. **Lichee RV Dock** — с программой генерации меандра на выводе GPIO 144 из лабораторной №9
5. **Преобразователь логических уровней**, размещённый на макетной плате

Установка и настройка Arduino IDE

1. Установите Arduino IDE из репозитория с помощью пакета *arduino*
2. Выберите правильную плату: **Инструменты** → **Плата** → **Arduino Uno**
3. Выберите правильный порт: **Инструменты** → **Порт** (например, COM3 или `/dev/ttyUSB0`)

Первая программа: «Моргающий светодиод»

Создайте новый скетч и напишите простейшую программу:

```
// Подключение светодиода к пину 13

void setup() {
  // Настройка пина как выхода
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  // Включить светодиод
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000); // Ждать 1 секунду

  // Выключить светодиод
  digitalWrite(LED_BUILTIN, LOW);
}
```

```

    delay(1000); // Ждать 1 секунду
}

```

`pinMode()` настраивает контакт как вход или выход, `digitalWrite()` устанавливает высокий или низкий логический уровень, а `delay()` приостанавливает выполнение на указанное количество миллисекунд.

Проверка

Проверка программы. После загрузки скетча встроенный светодиод должен включаться и выключаться с интервалом одна секунда.

Работа с прерываниями

Прерывания позволяют Arduino реагировать на внешние события без постоянной проверки в основном цикле.

Пины 2 и 3 на Arduino Uno могут быть использованы для работы с аппаратными прерываниями.

Программа с прерыванием:

```

// Пин для прерывания (порт 2 поддерживает прерывание INT0)
#define INTERRUPT_PIN 2

// Пин для светодиода
#define LED_PIN 5

// Счётчик прерываний
volatile int interruptCounter = 0;

void setup() {
    // Настройка пина светодиода
    pinMode(LED_PIN, OUTPUT);

    // Настройка пина прерывания как входа
    pinMode(INTERRUPT_PIN, INPUT);

    // Настройка прерывания:
    // - Пин: INTERRUPT_PIN (2)
    // - Функция обработки: handleInterrupt
    // - Режим: FALLING (срабатывает при переходе с HIGH на LOW)
    // Функция digitalPinToInterrupt преобразует номер пина в номер прерывания на
    ↪  нем

    // Таким образом пользовательская функция handleInterrupt привязывается к
    ↪  прерыванию
    // на пине INTERRUPT_PIN в режиме FALLING
    attachInterrupt(digitalPinToInterrupt(INTERRUPT_PIN), handleInterrupt,
    ↪  FALLING);

    // Инициализация последовательного порта для отладки

```

```

Serial.begin(115200);
Serial.println("Arduino готов к работе");
Serial.println("Ожидание прерываний от Lichee...");
}

void loop() {
    // Основной цикл может выполнять другие задачи
    digitalWrite(LED_PIN, HIGH);
    delay(500);
    digitalWrite(LED_PIN, LOW);
    delay(500);

    // Вывод счётчика прерываний
    static int lastCount = 0;
    if (interruptCounter != lastCount) {
        Serial.print("Получено прерываний: ");
        Serial.println(interruptCounter);
        lastCount = interruptCounter;
    }
}

// Функция обработки прерывания
void handleInterrupt() {
    interruptCounter++; // Увеличиваем счётчик
}

```

Задание

Ознакомьтесь с ключевыми понятиями, протестируйте элементарные программы и выполните следующие подзадачи.

Программа «Светофор»

Создайте программу, имитирующую бегущий огонёк на мезонинной плате путём мигания светодиодами, соединёнными с пинами 3, 5, 6, 9 и 10. Огонёк должен бежать от пина 3 к пину 10 и обратно, минуя пины, которые не соединены со светодиодами.

Программа «Повторитель меандра»

На Lichee RV Dock запустите программу для генерации меандра из лабораторной №9.

Опасность повреждения

Между Arduino Uno с логическими уровнями 5 В и Lichee RV Dock с уровнями 3,3 В обязателен преобразователь уровней и общий GND. Подайте Lichee 3,3 В на LV, Arduino 5 В на HV; соедините сигнал GPIO 144 Lichee с LV1, а соответствующий HV1 — с пином 2 Arduino. Не соединяйте выводы питания 3,3 В и 5 В друг с другом.

Соедините через логический преобразователь уровней пины в следующем соответствии:

- Lichee GPIO 144 (3,3 В) → LV1; HV1 → Arduino pin 2
- Lichee 3.3V → LV преобразователя
- Arduino 5V → HV преобразователя
- GND Lichee, Arduino и преобразователя → общий GND

Преобразователь логических уровней согласует сигналы устройств с разными логическими напряжениями. В этой схеме он преобразует сигнал Lichee с уровня 3,3 В в уровень, корректный для входа Arduino; двунаправленные каналы также защищают сторону Lichee при обмене в обратном направлении. На рисунке ниже приведён пример подключения Arduino к Lichee через макетную плату с преобразователем уровней:

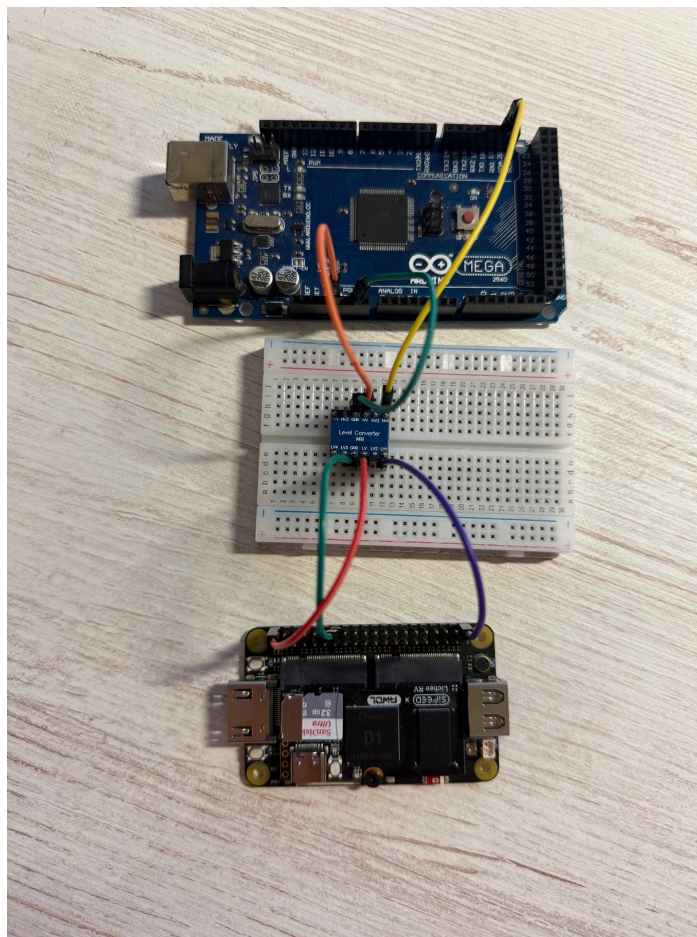


Рис. 1 – Пример подключения Lichee RV Dock к Arduino через преобразователь логических уровней

Затем напишите программу, которая на пине 4 повторяет генерируемый на Lichee меандр, работая с прерыванием на пине №2 в режиме CHANGE.

Анализ сигналов логическим анализатором:

Используя навык работы с логическим анализатором из лабораторной №9, проанализируйте сигналы:

1. **Подключите логический анализатор** к пину GPIO 144 на Lichee и пину 4 на Arduino
2. **Запустите PulseView**
3. **Наблюдайте прерывания** — сигналы от Lichee RV Dock
4. Убедитесь в корректности работы программы для Arduino
5. **Измерьте временные параметры** — длительность импульсов, интервалы и временную разницу между фронтами сигналов

Проверка

Проверка результата. На логическом анализаторе сигнал пина 4 Arduino должен повторять сигнал GPIO 144 Lichee. Измерьте задержку между соответствующими фронтами.

Контрольные вопросы

1. В чём ключевое отличие программирования микроконтроллера без ОС от разработки под операционную систему?
2. Какую роль выполняет загрузчик в Arduino?
3. Какие режимы срабатывания аппаратных прерываний доступны на ATmega328P и чем они отличаются?
4. Почему переменные, используемые и в обработчике прерывания, и в основном цикле, должны объявляться с ключевым словом `volatile`?
5. Зачем нужен преобразователь логических уровней при соединении Arduino с уровнями 5 В и Lichee RV Dock с уровнями 3,3 В?
6. Почему в обработчике прерывания не рекомендуется использовать функции `delay()` и `Serial.print()`?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- исходные тексты разработанных скетчей;
- схему или фотографию подключения Arduino и Lichee RV Dock;
- результаты измерения сигналов логическим анализатором;
- ответы на контрольные вопросы;
- краткий вывод.

Продемонстрируйте работу преподавателю и отправьте отчёт в согласованном формате до установленного срока.